

```
!pip install statsmodels -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

print("All libraries loaded!")
```

All libraries loaded!

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) | c107fa55-4...8d1fb0c.csv

c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv(text/csv) - 26651 bytes, last modified: 6/14/2026 - 100% done
Saving c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv to c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv

```
import os
print(os.listdir('/content'))
```

['.config', 'c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv', 'sample_data']

```
df = pd.read_csv('/content/c107fa55-45be-4f9c-8f61-088e88d1fb0c.csv')
print(df.shape)
print(df.head())
print(df.dtypes)
print(df.isnull().sum())
```

```
(572, 5)
      TV      Radio  Social Media Influencer      Sales
0  Low   3.518070    2.293790      Micro  55.261284
1  Low   7.756876    2.572287      Mega  67.574904
2  High  20.348988    1.227180      Micro  272.250108
3  Medium 20.108487    2.728374      Mega  195.102176
4  High  31.653200    7.776978      Nano  273.960377
```

```
TV      object
Radio    float64
Social Media float64
Influencer object
Sales    float64
dtype: object
TV      0
Radio   0
Social Media 0
Influencer 0
Sales      0
dtype: int64
```

```
# Convert TV from categorical to numerical
df['TV'] = df['TV'].map({'Low': 1, 'Medium': 2, 'High': 3})

# Check
print(df.head())
print(df.dtypes)
```

```
      TV      Radio  Social Media Influencer      Sales
0  1  3.518070    2.293790      Micro  55.261284
1  1  7.756876    2.572287      Mega  67.574904
2  3  20.348988    1.227180      Micro  272.250108
3  2  20.108487    2.728374      Mega  195.102176
4  3  31.653200    7.776978      Nano  273.960377
TV      int64
Radio    float64
Social Media float64
Influencer object
Sales    float64
dtype: object
```

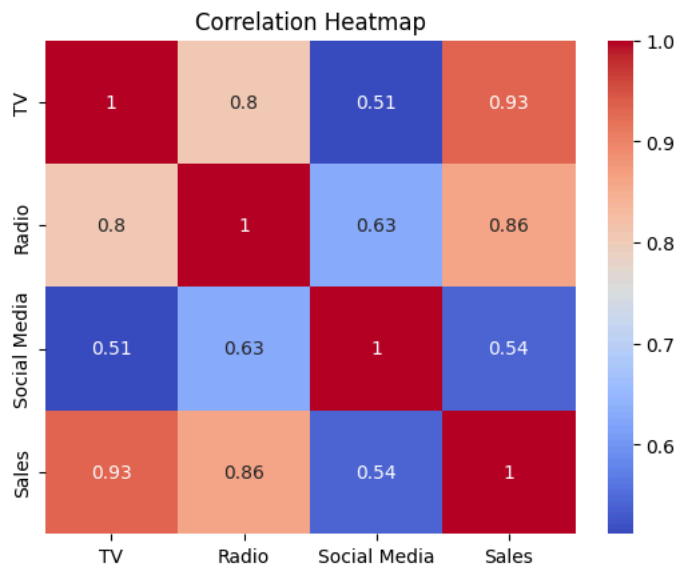
```
# Correlation matrix
correlation = df[['TV', 'Radio', 'Social Media', 'Sales']].corr()
print(correlation)
```

```
# Heatmap
sns.heatmap(correlation, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

```

      TV      Radio  Social Media   Sales
TV      1.000000  0.803377    0.511758  0.933169
Radio    0.803377  1.000000    0.629941  0.858036
Social Media 0.511758 0.629941    1.000000  0.542048
Sales     0.933169 0.858036    0.542048  1.000000

```



```
# Variance Inflation Factor
X = df[['TV', 'Radio', 'Social Media']]
X = sm.add_constant(X)

vif_data = pd.DataFrame()
vif_data['Feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(len(X.columns))]
print(vif_data)
```

	Feature	VIF
0	const	7.040920
1	TV	2.820621
2	Radio	3.451590
3	Social Media	1.658144

```
# Build Multiple Linear Regression model
model = smf.ols(formula='Sales ~ TV + Radio + Q("Social Media")', data=df).fit()
print(model.summary())
```

OLS Regression Results

```

=====
Dep. Variable:      Sales      R-squared:      0.904
Model:              OLS      Adj. R-squared:    0.903
Method:             Least Squares      F-statistic:    1781.
Date:               Sun, 14 Jun 2026      Prob (F-statistic): 2.10e-288
Time:               06:41:39      Log-Likelihood:   -2714.2
No. Observations:   572      AIC:              5436.
Df Residuals:       568      BIC:              5454.
Df Model:           3
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-12.2889	3.099	-3.966	0.000	-18.375	-6.202
TV	77.3227	2.456	31.486	0.000	72.499	82.146
Radio	2.9792	0.234	12.746	0.000	2.520	3.438
Q("Social Media")	-0.1577	0.672	-0.235	0.815	-1.478	1.163

```

=====
Omnibus:            60.101      Durbin-Watson:      1.874
Prob(Omnibus):      0.000      Jarque-Bera (JB):    17.942
Skew:               0.050      Prob(JB):            0.000127
Kurtosis:           2.138      Cond. No.            61.5
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
# Diagnostic plots
fitted = model.fittedvalues
residuals = model.resid

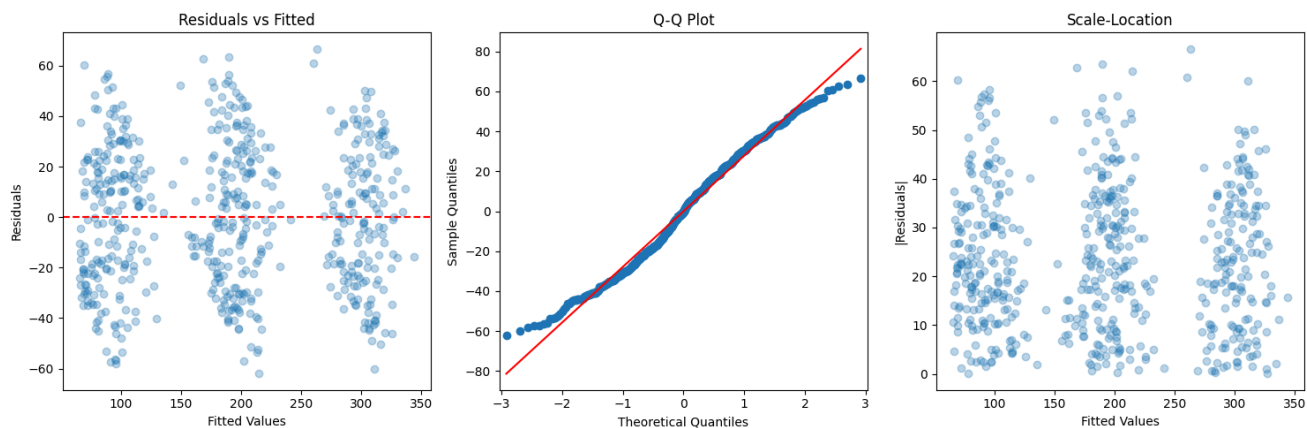
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Plot 1: Residuals vs Fitted (Linearity)
axes[0].scatter(fitted, residuals, alpha=0.3)
axes[0].axhline(0, color='red', linestyle='--')
axes[0].set_title('Residuals vs Fitted')
axes[0].set_xlabel('Fitted Values')
axes[0].set_ylabel('Residuals')

# Plot 2: Q-Q Plot (Normality)
sm.qqplot(residuals, line='s', ax=axes[1])
axes[1].set_title('Q-Q Plot')

# Plot 3: Scale-Location (Homoscedasticity)
axes[2].scatter(fitted, abs(residuals), alpha=0.3)
axes[2].set_title('Scale-Location')
axes[2].set_xlabel('Fitted Values')
axes[2].set_ylabel('|Residuals|')

plt.tight_layout()
plt.show()
```



```
print("=" * 60)
print("MULTIPLE LINEAR REGRESSION - KEY FINDINGS")
print("=" * 60)
print()
print("REGRESSION EQUATION:")
print("Sales = -12.29 + 77.32*TV + 2.98*Radio + (-0.16)*Social Media")
print()
print("ADJUSTED R-SQUARED: 0.903")
print("All 3 channels explain 90.3% of Sales variation")
print()
print("COEFFICIENT INTERPRETATION:")
print("TV:          +77.32 sales per unit increase (p=0.000) ✅")
print("Radio:         +2.98 sales per unit increase (p=0.000) ✅")
print("Social Media:  -0.16 sales per unit increase (p=0.815) ❌")
print()
print("MULTICOLLINEARITY CHECK (VIF):")
print("TV VIF:          2.82 - Acceptable")
print("Radio VIF:       3.45 - Acceptable")
print("Social Media VIF: 1.66 - Acceptable")
print()
print("RECOMMENDATION:")
print("1. Prioritize TV advertising - strongest ROI ($77.32 per unit)")
print("2. Maintain Radio spend - significant positive impact")
```

```
print("3. Reduce Social Media budget - not statistically significant")
print("=" * 60)
```

=====

MULTIPLE LINEAR REGRESSION - KEY FINDINGS

=====

REGRESSION EQUATION:

Sales = -12.29 + 77.32*TV + 2.98*Radio + (-0.16)*Social Media

ADJUSTED R-SQUARED: 0.903

All 3 channels explain 90.3% of Sales variation

COEFFICIENT INTERPRETATION:

TV:	+77.32 sales per unit increase	(p=0.000)	✓
Radio:	+2.98 sales per unit increase	(p=0.000)	✓
Social Media:	-0.16 sales per unit increase	(p=0.815)	✗

MULTICOLLINEARITY CHECK (VIF):

TV VIF: 2.82 - Acceptable

Radio VIF: 3.45 - Acceptable

Social Media VIF: 1.66 - Acceptable

RECOMMENDATION:

1. Prioritize TV advertising - strongest ROI (\$77.32 per unit)
2. Maintain Radio spend - significant positive impact
3. Reduce Social Media budget - not statistically significant

=====

Start coding or [generate](#) with AI.